دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

برنامه‌سازی پیشرفته و کارگاه

شی‌گرایی (ارث‌بری)

نگارش:

سام قربانی، آرمان حسینی، محمدحسین هاشمی، کیانا پهلوان، کیانا رضوی، اریا فیض بخش، مهدی جعفری، مارال مهاجر

استاد درس:

دکتر روح‌الله احمدیان

دانشکده ریاضی و علوم کامپیوتر

دانشگاه صنعتی امیرکبیر

بهار ۱۴۰۵

فهرست مطالب

۱ شی‌گرایی ۲ . . . . . ۲
۱.۱ مقدمه ۲
۲.۱ ارث‌بری ۲
۱.۲.۱ توصیف یک مشکل ۲
۲.۲.۱ راه‌حل ۵
۳.۲.۱ subclass ها و superclass ها ۸
۳.۱ کلاس‌های Abstract. ۱۰

فصل ۱ شی‌گرایی (ارث‌بری)

۱.۱ مقدمه

ارث‌بری، از پایه‌ای‌ترین مفاهیم شی‌گرایی است که به کلاس‌های مشابه، این امکان را می‌دهد که متدها و فیلدهای یکسان را در یک class جدید تعریف کنند. با استفاده از ارث‌بری، می‌توانیم به طراحی‌ای بهتر برای برنامه‌مان برسیم به طوری که کدهای تکراری کمی داشته باشیم و درنتیجه، برنامه maintainable تر باشد.

حتماً کدهای این داک رو اجرا کنید و هر سوالی که داشتید از تدریس‌یارها بپرسید، در موردش سرچ کنید و از ChatGPT راجع بهش اطلاعات بخواید. ارث‌بری از مفاهیم مهم درس است که یادگیری آن اهمیت زیادی دارد و هسته‌ی اصلی شی گرایی را تشکیل می‌دهد.

۲.۱ ارث‌بری

۱.۲.۱ توصیف یک مشکل

در IntelliJ یک پروژهٔ جدید ایجاد کنید و یک پکیج به اسم `vehicle` درست کنید. در `vehicle` دو فایل جدید به اسم `Car.java` و `Bike.java` درست کنید و داخل اون‌ها، کلاس‌های `Car` و `Bike` رو ایجاد کنید:

```
package Vehicle;

public class Bike {
    public String brand;
    public int speed;
    public boolean hasCarrier;

    public Bike(String brand, int speed, boolean hasCarrier) {
        this.brand = brand;
        this.speed = speed;
        this.hasCarrier = hasCarrier;
    }

    public void accelerate(int speed) {
        this.speed += speed;
    }
```

```
    public void showBikeInfo() {
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
        System.out.println("Has Carrier: " + hasCarrier);
    }
}
```

```
package Vehicle;

public class Car {
    public String brand;
    public int speed;
    public int doors;

    public Car(String brand, int speed, int doors) {
        this.brand = brand;
        this.speed = speed;
        this.doors = doors;
    }

    public void accelerate(int speed) {
        this.speed += speed;
    }

    public void showCarInfo() {
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
        System.out.println("Doors: " + doors);
    }
}
```

بعد از این کار، کد Main رو به کد زیر تغییر دهید و آن را اجرا کنید تا مطمئن شوید که همه چیز درست کار می‌کند:

```
public static void main(String[] args) {
    Car myCar = new Car("Toyota", 180, 4);
    myCar.showCarInfo();

    System.out.println();

    Bike myBike = new Bike("Yamaha", 120, true);
```

```
    myBike.showBikeInfo();
}
```

بعد از اجرای کد، باید چنین چیزی ببینید:

```
Brand: Toyota, Speed: 180 km/h
Doors: 4

Brand: Yamaha, Speed: 120 km/h
Has Carrier: true
```

حالا بیاید راجع به شباهت‌های این دو کلاس صحبت کنیم. ظاهر این کلاس‌ها خیلی شبیه به هم هستند. هر دو فیلدهایی به اسم brand و speed دارند، با متدی به اسم accelerate سرعت وسیلهٔ نقلیه را زیاد می‌کنند و همینطور متدهایی دارند که اطلاعات آن‌ها را نمایش می‌دهند. در واقع، اگر یک بار به کد نگاه کنید، می‌بینید کلاس‌های Bike و Car تفاوت چندانی ندارند! حالا بیاید یک کلاس جدید مثل MotorCycle در فایل جدیدی به اسم MotorCycle.java بنویسیم:

```
package Vehicle;

public class MotorCycle {
    public String brand;
    public int speed;
    public boolean hasSidecar;

    public MotorCycle(String brand, int speed, boolean hasSidecar) {
        this.brand = brand;
        this.speed = speed;
        this.hasSidecar = hasSidecar;
    }

    public void accelerate(int speed) {
        this.speed += speed;
    }

    public void showMotorCycleInfo() {
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
        System.out.println("Has Sidecar: " + hasSidecar);
    }
}
```

این کد را هم امتحان کنید:

```
MotorCycle myMotorCycle = new MotorCycle("Benz", 120, true);
myMotorCycle.showBikeInfo();
```

باید خروجی‌ای مثل زیر ببینید:

```
Brand: Benz, Speed: 120 km/h
Has Sidecar: true
```

کلاس جدیدی که ساختیم هم کار می‌کند، ولی باز هم شبیه به دو کلاس قبلی است! همانطور که می‌بینید هر وسیلهٔ نقلیه‌ای که بخواهیم اضافه کنیم، باید کدهای کلاس‌های قبلی را تکرار کنیم. همانطور که در داک clean code گفته شد، ما در برنامه‌نویسی، سعی می‌کنیم از کدهای تکراری دوری کنیم.

۲.۲.۱ راه‌حل

راه حل این مشکل این است که **ویژگی‌های مشترک تمام این کلاس‌ها را، داخل کلاس جدیدی به اسم** `Vehicle` **بنویسیم و بعد، با استفاده از ارث‌بری، آن ویژگی‌ها را به همه این کلاس‌ها منتقل کنیم**. حالا این که «ویژگی‌های یک کلاس را به کلاسی دیگر منتقل کنیم» یعنی چه؟ بیاید تا با کد زدن ببینیم. توی فایل `Vehicle.java`، کلاس `Vehicle` رو ایجاد کنید. این کلاس، به جای این که نمایندهٔ ماشین و موتور و دوچرخه باشد، نمایندهٔ یک «وسیلهٔ نقلیه» است و ویژگی‌های مشترک تمام کلاس‌های قبل را داخل خود نگه می‌دارد:

```
package Vehicle;

public class Vehicle {
    public String brand;
    public int speed;

    public Vehicle(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    public void accelerate(int speed) {
        this.speed += speed;
    }

    public void showInfo() {
```

```
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
    }
}
```

می‌بینید که فیلدها و متدهای مشترک تمام وسایل نقلیه، یعنی brand ، speed ، accelerate و غیره، همگی داخل این کلاس هستند. حالا بیاید کد کلاس Car رو عوض کنیم:

```
public class Car extends Vehicle {
    public int doors;

    public Car(String brand, int speed, int doors) {
        super(brand, speed);
        this.doors = doors;
    }

    public void showCarInfo() {
        showInfo();
        System.out.println("Doors: " + doors);
    }
}
```

در این کد، شما برای اولین بار با کلیدواژهٔ extends آشنا می‌شوید. این کلیدواژه در جاوا برای تعیین ارث‌بری استفاده می‌شود. در کد بالا کلاس Car ، تمام فیلدها و متدهای کلاس Vehicle رو دارد. مثلا در خط اول متد showCarInfo() ، می‌بینید که متد showInfo() صدا زده شده اما Car چنین متدی ندارد. در واقع، این متد در کلاس Vehicle تعریف شده و چون Car از Vehicle ارث‌بری کرده، پس می‌تواند از متدها و فیلدهای آن استفاده بکند!

حالا در main کد زیر را اجرا کنید تا مطمئن شوید همه چیز درست کار می‌کند:

```
public static void main(String[] args) {
    Car myCar = new Car("Toyota", 180, 4);
    myCar.showCarInfo();

    System.out.println();

    myCar.accelerate(5);
    myCar.showCarInfo();
}
```

بعد از اجرای این کد، باید خروجی زیر را ببینید:

```
Brand: Toyota, Speed: 180 km/h
Doors: 4

Brand: Toyota, Speed: 185 km/h
Doors: 4
```

می‌بینید که حتی توانستیم از متد accelerate هم برای myCar استفاده کنیم، در حالی که این متد اصلا در Car وجود ندارد! این موضوع فقط به خاطر این است که Car از Vehicle ارث‌بری می‌کند، و به همین دلیل به تمام فیلدها و متدهای آن دسترسی دارد.

حالا کد کلاس‌های Bike و MotorCycle را به شکل زیر تغییر دهید تا آن‌ها هم از کلاس Vehicle ارث‌بری کنند:

```java
package Vehicle;

public class MotorCycle extends Vehicle {
    boolean hasSidecar;

    public MotorCycle(String brand, int speed, boolean hasSidecar) {
        super(brand, speed);
        this.hasSidecar = hasSidecar;
    }

    public void showBikeInfo() {
        showInfo();
        System.out.println("Has Sidecar: " + hasSidecar);
    }
}
```

```java
package Vehicle;

public class Bike extends Vehicle {
    public boolean hasCarrier;

    public Bike(String brand, int speed, boolean hasCarrier) {
        super(brand, speed);
        this.hasCarrier = hasCarrier;
    }

    public void showMotorCycleInfo() {
```

```
        showInfo();
        System.out.println("Has Carrier: " + hasCarrier);
    }
}
```

می‌بینید که حجم کدهای تکراری خیلی کمتر شدند. با کد زیر، این کلاس‌های جدید را تست کنید:

```
public static void main(String[] args) {
    Bike myBike = new Bike("Yamaha", 120, true);
    myBike.showBikeInfo();

    System.out.println();

    myBike.accelerate(5);
    myBike.showBikeInfo();

    System.out.println();

    MotorCycle myMotorcycle = new MotorCycle("Harley-Davidson", 160, false);
    myMotorcycle.showMotorCycleInfo();
}
```

باید خروجی زیر را ببینید:

```
Brand: Yamaha, Speed: 120 km/h
Has Carrier: true

Brand: Yamaha, Speed: 125 km/h
Has Carrier: true

Brand: Harley-Davidson, Speed: 160 km/h
Has Sidecar: false
```

می‌بینید که همهٔ این کلاس‌ها، مثل کلاس Car می‌توانند از فیلدهای brand و speed و متدهای accelerate() و showInfo() استفاده کنند.

۳.۲.۱ subclass ها و superclass ها

در کد بالا، به کلاس Vehicle که از آن ارث‌بری شده superclass یا کلاس والد می‌گویند. همچنین به کلاس‌های Car ، Bike و MotorCycle که از کلاس Vehicle ارث‌بری کردند subclass یا کلاس‌های

فرزند می‌گویند. هر کلاس می‌تواند از حداکثر یک کلاس به صورت مستقیم ارث‌بری کند. یعنی هر کلاس حداکثر یک والد دارد.

همینطور که می‌بینید در خط اول constructor های subclass ها، از عبارتی به اسم super استفاده شده است:

```
public Bike(String brand, int speed, boolean hasCarrier) {
    super(brand, speed);
    this.hasCarrier = hasCarrier;
}
```

هر زمان از کلید واژه جدید super استفاده می‌کنیم، یعنی قصد داریم از فیلدها، متدها یا کانستراکتورهای کلاس پدر استفاده کنیم. مثلا در این‌جا، با استفاده از کلیدواژهٔ super ، نشان دادیم که می‌خواهیم کانستراکتور زیر از کلاس پدر را استفاده کنیم تا فیلدهای brand و speed برای آبجکت جدید مقداردهی شوند:

```
public Vehicle(String brand, int speed) {
    this.brand = brand;
    this.speed = speed;
}
```

یا مثلا، در متد showBikeInfo() ، می‌توانیم با استفاده از super ، دقیق‌تر مشخص کنیم که متد showInfo ی کلاس Vehicle صدا زده شود:

```
public void showBikeInfo() {
    super.showInfo();
    System.out.println("Has Carrier: " + hasCarrier);
}
```

همچنین با استفاده از این کلیدواژه، شما می‌توانید به فیلدهای کلاس پدر هم دسترسی داشته باشید:

```
public void showBikeInfo() {
    super.showInfo();
    System.out.println("Has Carrier: " + hasCarrier);

    System.out.println("Speed: " + super.speed);
    System.out.println("Brand: " + super.brand);
}
```

البته نتیجه کپی کردن کد بالا به جای کد `ShowBikeInfo()` این است که سرعت و برند دوچرخه‌ها دو بار چاپ می‌شوند! هدف از نشان دادن کد بالا این بود که ببینید با کلیدواژهٔ super ، به تمام ویژگی‌های کلاس پدر دسترسی خواهید داشت. در ادامه این داک، متدهای کلاس پدر را Override می‌کنید و بیشتر به اهمیت این موضوع پی می‌برید.

۳.۱ کلاس‌های Abstract

فرض کنید می‌خواهید یک برنامه بنویسید تا حقوق کارمندان را حساب کند. سازمان شما دو نوع کارمند دارد:

- **کارمندهای تمام‌وقت:** این کارمندها، ماهیانه مبلغ ثابتی حقوق می‌گیرند. برای مثال ممکن است هر ماه ۳۰ میلیون تومان حقوق دریافت کنند.
- **کارمندهای قراردادی:** این کارمندها، براساس ساعاتی که در ماه در شرکت بودند حقوق می‌گیرند. برای مثال اگر ساعتی ۱ میلیون تومان دریافت کنند ، اگر ۲۰ ساعت در شرکت باشند ۲۰ میلیون و اگر ۴۵ ساعت باشند، ۴۵ میلیون حقوق دریافت می کنند.

پکیج `company` را برای درست کردن کلاس‌ها ایجاد کنید. دوتا کلاس `FullTimeEmployee` و `Contractor` را در این پکیج ایجاد کنید:

```
package company;

public class Contractor {
    public String name;
    public int id;
    public double hourlyRate;
    public int hoursWorked;

    public Contractor(String name, int id, double hourlyRate, int
↪   hoursWorked) {
        this.name = name;
        this.id = id;
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }

    public void showDetails() {
        System.out.println("ID: " + id + ", Name: " + name);
    }

    public double calculateSalary() {
        return hourlyRate * hoursWorked;
```

```
    }
}
```

```
package company;

public class FullTimeEmployee {
    public String name;
    public int id;
    public double monthlySalary;

    public FullTimeEmployee(String name, int id, double monthlySalary) {
        this.name = name;
        this.id = id;
        this.monthlySalary = monthlySalary;
    }

    public void showDetails() {
        System.out.println("ID: " + id + ", Name: " + name);
    }

    public double calculateSalary() {
        return monthlySalary;
    }
}
```

در این دو کلاس، موارد مشترک زیادی می‌شود دید. مثل فیلدهای name و id و متد showDetails که کاملا یکسان هستند. این شباهت، این حس را ایجاد می کند که باید از inheritance استفاده کنید. برای این کار، شما کلاس Employee رو تعریف می‌کنید:

```
package company;

public class Employee {
    public String name;
    public int id;

    public Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }
```

```
    public void showDetails() {
        System.out.println("ID: " + id + ", Name: " + name);
    }
}
```

و کدتون رو به شکلی تغییر دهید که کلاس‌های FullTimeEmployee و Contractor هر دو از آن ارث‌بری کند. کلاس‌های زیر را در پکیج company ایجاد کنید یا تغییر بدهید:

```
package company;

public class Contractor extends Employee {
    public double hourlyRate;
    public int hoursWorked;

    public Contractor(String name, int id, double hourlyRate, int
↪   hoursWorked) {
        super(name, id);
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }

    public double calculateSalary() {
        return hourlyRate * hoursWorked;
    }
}
```

```
package company;

public class FullTimeEmployee extends Employee {
    public double monthlySalary;

    public FullTimeEmployee(String name, int id, double monthlySalary) {
        super(name, id);
        this.monthlySalary = monthlySalary;
    }

    public double calculateSalary() {
        return monthlySalary;
    }
}
```

می‌بینید که کد تمیزتری شد. حالا بیاید در متد main ، یک سری کارمند تعریف کنیم و مجموع حقوقی که باید بدیم را حساب کنیم:

```
public static void main(String[] args) {
    FullTimeEmployee ali = new FullTimeEmployee("Ali", 1, 10_000);
    Contractor gholi = new Contractor("Gholi", 2, 1_000, 100);
    Contractor mamad = new Contractor("Mamad", 3, 1_000, 25);

    ArrayList<FullTimeEmployee> fullTimeEmployees = new ArrayList<>();
    ArrayList<Contractor> contractors = new ArrayList<>();

    fullTimeEmployees.add(ali);
    contractors.add(mamad);
    contractors.add(gholi);

    double sumOfSalary = 0;
    for (FullTimeEmployee employee : fullTimeEmployees) {
        sumOfSalary += employee.calculateSalary();
    }
    for (Contractor contractor : contractors) {
        sumOfSalary += contractor.calculateSalary();
    }

    System.out.println("Salary this month: " + sumOfSalary);
}
```

این کد را یکبار بخوانید تا بفهمید چه کاری انجام می‌دهد. آن را اجرا کنید تا نتیجه زیر را ببینید:

```
Salary this month: 135000.0
```

یک مشکل در این کد وجود دارد. فرض کنید رئیس می‌خواهد که نوع جدیدی از کارمند داشته باشیم. مثلا فرض کنید که می‌خواهیم حقوق مدیران به شکل متفاوتی محاسبه شود. کلاس Manager را اضافه می‌کنیم و بعد از این کار، برای محاسبهٔ حقوق حلقهٔ زیر را به main اضافه می‌کنیم:

```
for (Manager manager : managers) {
    sumOfSalary += manager.calculateSalary();
}
```

بعدتر تصمیم می‌گیرید که حقوق کله گنده‌ها به شکل متفاوتی محاسبه شود! شما کلاس KaleGonde را برای آن‌ها درست می‌کنید و برای حساب کردن حقوق حلقهٔ زیر را اضافه می‌کنید:

```
for (KaleGonde kaleGonde : kaleGondeha) {
    sumOfSalary += kaleGonde.calculateSalary();
}
```

کد نهایی چنین چیزی خواهد بود:

```
double sumOfSalary = 0;
for (FullTimeEmployee employee : fullTimeEmployees) {
    sumOfSalary += employee.calculateSalary();
}
for (Contractor contractor : contractors) {
    sumOfSalary += contractor.calculateSalary();
}
for (Manager manager : managers) {
    sumOfSalary += manager.calculateSalary();
}
for (KaleGonde kaleGonde : kaleGondeha) {
    sumOfSalary += kaleGonde.calculateSalary();
}
```

حدس بزنید مشکل این کد کجاست؟ بله دقیقا. این حلقه‌ها خیلی مشابه هستند و ما کدهای شبیه به هم را ترجیح نمی‌دهیم! یک راه حل خوب برای این موضوع، این هست که کلاس Employee ، که پدر همهٔ کلاس‌های برنامه است را abstract کنیم. در ابتدا کد را ببینیم:

```
package company;

public abstract class Employee {
    public String name;
    public int id;

    public Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public void showDetails() {
        System.out.println("ID: " + id + ", Name: " + name);
    }
```

```
    public abstract double calculateSalary();
}
```

با استفاده از کلیدواژهٔ abstract در خط سوم این کد، کلاس Employee را به یک Abstract Class تبدیل کردیم. می‌بینید که در این کلاس، یک اتفاق عجیب افتاده. متد calculateSalary() تعریف شده است، ولی هیچ چیزی در {} برای پیاده‌سازی آن نیامده. این یعنی کلاس Employee، پیاده‌سازی این متد را به subclass های خود واگذار کرده.

در واقع، کلاس Employee به شما می گوید «هر کارمند، یک متد به اسم calculateSalary() دارد که هیچ چیزی ورودی نمی‌گیرد و خروجی آن، double خواهد بود. پیاده‌سازی این متد بسته به نوع کارمند (FullTime، Contractor، Manager یا KaleGonde بودن) متفاوت است، ولی هر کلاس، قطعا این متد را دارد و شما می‌توانید آن را فراخوانی کنید.».

شاید مقداری گیج کننده باشد، بیاید تا کد main را یک مقدار عوض کنیم تا این مطلب برایتان به خوبی مفهوم شود:

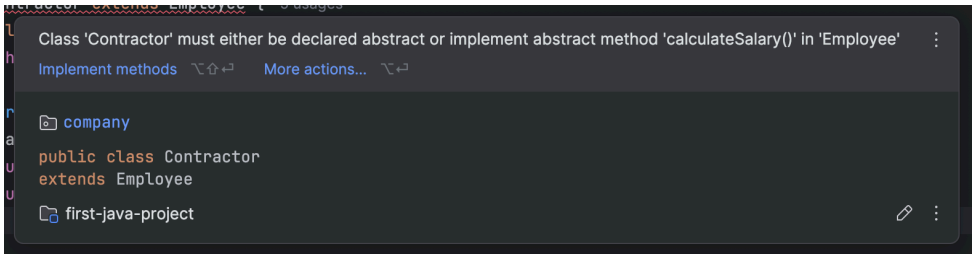
```
public static void main(String[] args) {
    FullTimeEmployee ali = new FullTimeEmployee("Ali", 1, 10_000);
    Contractor gholi = new Contractor("Gholi", 2, 1_000, 100);
    Contractor mamad = new Contractor("Mamad", 3, 1_000, 25);

    ArrayList<Employee> allEmployees = new ArrayList<>();

    allEmployees.add(ali);
    allEmployees.add(mamad);
    allEmployees.add(gholi);

    double sumOfSalary = 0;
    for (Employee employee : allEmployees) {
        sumOfSalary += employee.calculateSalary();
    }

    System.out.println("Salary this month: " + sumOfSalary);
}
```

کد فعلی main را نگاه کنید و تفاوت‌های آن را با کد قبلی ببینید. می‌بینید که آرایه ما از جنس Employeeاست، و آن را با FullTimeEmployee ها و Contractor ها پر کردیم. از اون‌جایی که می‌دانیم هر Employee فارغ از نوع قطعا یک متد calculateSalary() دارد، پس فقط یک حلقه کافیست که درآمد همهٔ کارکنان را جمع بزنیم. اگر این کد اجرا شود، خروجی قبل را می‌بینید:

```
Salary this month: 135000.0
```

کار abstract class ها دقیقا همین چیزی است که این‌جا دیدید. وقتی در کلاس پدر، نمی‌دانیم که کلاس‌های فرزند یک متد را به چه شکل پیاده‌سازی می‌کنند، ولی می‌دانیم که حتماً آن را پیاده‌سازی می‌کنند، از abstract class استفاده می‌کنیم.

دقت کنید که دوتا نکته در abstract class ها بسیار مهم است. اول این که کلاس‌های فرزند، حتما باید متدهای abstract ای که کلاس پدر تعریف کرده پیاده‌سازی کنند، چرا که در غیر این صورت به خطا می‌خوریم. یک بار متد `calculateSalary` را از `Contractor` پاک کنید تا خطایش را ببینید:



این خطا به شما می‌گوید که «کلاس `Contractor` یا باید خودش abstract باشد (و در نتیجه پیاده‌سازی متدهای abstract را به فرزندانش بسپرد)، یا باید متد `calculateSalary` را پیاده‌سازی کند».

نکتهٔ دوم این است که چون کلاس‌های abstract پیاده‌سازی کاملی ندارند، شما نمی‌توانید از این کلاس‌ها مستقیما `new` کنید. کد زیر به خطا می‌خورد:

```
Employee javad = new Employee("Javad", 4);
```

کلاس‌های abstract ، فقط ظرفی برای subclass های خود هستند و نباید از آنها مستقیم آبجکت ساخته بشود. در عوض، چیزی مثل کد زیر کاملا درست است:

```
FullTimeEmployee ali = new FullTimeEmployee("Ali", 1, 10_000);
Employee ali2 = ali;
```